Solution: Problem Challenge 3: Count of Structurally Unique Binary Search Trees

Problem Statement

Given a number ‘n’, write a function to return the count of structurally unique Binary Search Trees (BST) that can store values 1 to ‘n’.

Example 1:

```
Input: 2
Output: 2
Explanation: As we saw in the previous problem, there are 2 unique BSTs storing
numbers from 1-2.
```

Example 2:

```
Input: 3
Output: 5
Explanation: There will be 5 unique BSTs that can store numbers from 1 to 3.
```

Solution

This problem is similar to `Structurally Unique Binary Search Trees`. Following a similar approach, we can iterate from 1 to ‘n’ and consider each number as the root of a tree and make two recursive calls to count the number of left and right sub-trees.

Code

Here is what our algorithm will look like:



```
//   TreeNode(int x) {
//     val = x;
//   }
// };

class Solution {
  public int countTrees(int n) {
    if (n <= 1)
      return 1;
    int count = 0;
    for (int i = 1; i <= n; i++) {
      // making 'i' root of the tree
      int countOfLeftSubtrees = countTrees(i - 1);
      int countOfRightSubtrees = countTrees(n - i);
      count += (countOfLeftSubtrees * countOfRightSubtrees);
    }
```

Run Code Result | Stdout

You must run your code first

Execute

Time complexity

The time complexity of this algorithm will be exponential and will be similar to Balanced Parentheses. Estimated time complexity will be $O(n * 2^n)$ but the actual time complexity $O(4^n/n)$ is bounded by the Catalan number and is beyond the scope of a coding interview. See more details here.

Space complexity

The space complexity of this algorithm will be exponential too, estimated $O(2^n)$ but the actual will be $O(4^n/n)$.

Memoized version

Our algorithm has overlapping subproblems as our recursive call will be evaluating the same sub-expression multiple times. To resolve this, we can use memoization and store the intermediate results in a HashMap. In each function call, we can check our map to see if we have already evaluated this sub-expression before. Here is the memoized version of our algorithm:

Java

```
import java.util.*;

// class TreeNode {
//   int val;
//   TreeNode left;
```

```
6   //   TreeNode right;
7
8   //   TreeNode(int x) {
9   //     val = x;
10  //   }
11  // };
12
13  class Solution {
14    Map<Integer, Integer> map = new HashMap<>();
15
16    public int countTrees(int n) {
17      if (map.containsKey(n))
18        return map.get(n);
19
20      if (n <= 1)
21        return 1;
22      int count = 0;
23      for (int i = 1; i <= n; i++) {
24        // making 'i' root of the tree
```

Run Code Result | Stdout

You must run your code first

Execute

The time complexity of the memoized algorithm will be $O(n^2)$, since we are iterating from '1' to 'n' and ensuring that each sub-problem is evaluated only once. The space complexity will be $O(n)$ for the memoization map.

← Previous

Problem Challenge 3: Count Of Structurally Unique Binary Search Trees (Hard)

Next →

Introduction To Modified Binary Search Pattern

Mark as Completed